

Prompt:

8/17/2026, 10:18:47 PM

lanjut Tahap AD

Response:

8/17/2026, 10:18:48 PM

Tahap AD — Trade Opportunity, First-Passage & Optimal TP/SL Engine

Baik. Tahap AD adalah salah satu bagian **paling penting** dari seluruh arsitektur kita, karena di sinilah sistem mulai berubah dari:

"model memprediksi return"

menjadi:

"model menentukan apakah sebuah trade secara matematis layak dilakukan, di mana entry-nya, di mana SL/TP-nya, dan kapan struktur trade tersebut tidak lagi optimal."

Pada Tahap AC kita sudah memperoleh informasi microstructure:

$$M_t = \{OBI_t, OFI_t, Spread_t, Depth_t, p_{fill}, Slippage_t, Impact_t, \dots\}.$$

Sekarang kita gabungkan dengan:

$$\hat{\mu}_t, \quad \hat{\sigma}_t, \quad \hat{\Sigma}_t, \quad Regime_t$$

untuk membangun **distribusi perjalanan harga setelah entry**.

AD.1. Masalah yang sebenarnya ingin kita selesaikan

Misalkan sistem menemukan:

$$\hat{\mu}_t > 0.$$

Secara sederhana kita mungkin berkata:

Buy.

Tetapi itu belum menjawab:

1. Berapa harga entry optimal?
2. Berapa jauh SL?
3. Berapa jauh TP?
4. Berapa probabilitas TP tercapai?
5. Berapa probabilitas SL tercapai?
6. Mana yang tercapai lebih dahulu?
7. Berapa lama trade diperkirakan bertahan?
8. Apakah expected value positif?
9. Apakah trade masih layak setelah memperhitungkan fee/slippage?
10. Jika posisi sudah berjalan, apakah TP/SL perlu diubah?

Maka objek matematis kita bukan lagi sekadar:

$$R_{t+1}.$$

Kita membutuhkan **price path**:

$$\{P_{t+s}\}_{s \geq 0}.$$

AD.2. Dari return menjadi stochastic process

Kita mulai dari log-price:

$$X_t = \ln P_t.$$

Return kontinu:

$$dX_t = \mu_t dt + \sigma_t dW_t.$$

Jika kondisi pasar relatif stationary untuk horizon pendek, kita dapat menggunakan pendekatan:

$$dX_t = \mu dt + \sigma dW_t.$$

Ini adalah geometric Brownian motion pada harga:

$$dP_t = \mu P_t dt + \sigma P_t dW_t.$$

Tetapi untuk sistem kita, GBM **bukan model final**.

Ia hanya baseline matematis.

AD.3. Mengapa GBM saja tidak cukup?

Pasar nyata memiliki:

- volatility clustering,
- fat tails,
- jumps,
- autocorrelation pada order flow,
- regime changes,
- asymmetric response,
- liquidity shocks.

Karena itu nantinya kita dapat mengganti:

$$\mu, \sigma$$

dengan:

$$\mu_t = f(F_t),$$

$$\sigma_t = g(F_t).$$

Namun untuk menurunkan fondasi first-passage secara analitik, kita mulai dari proses sederhana terlebih dahulu.

AD.4. Return conditional terhadap informasi saat ini

Pada waktu t :

$$\mu_t = E[dX_t | F_t]/dt.$$

dan:

$$\sigma_t^2 = \text{Var}[dX_t | \mathcal{F}_t]/dt.$$

Jadi μ_t dan σ_t bukan konstanta universal.

Mereka merupakan output dari model kita.

AD.5. Definisi entry

Misalkan entry price:

$$P_E.$$

Dalam log-space:

$$X_E = \ln P_E.$$

Kita kemudian menentukan:

$$P_{SL} < P_E < P_{TP}$$

untuk posisi long.

Dalam log-space:

$$X_{SL} < X_E < X_{TP}.$$

Definisikan jarak:

$$a = X_E - X_{SL} > 0$$

dan:

$$b = X_{TP} - X_E > 0.$$

AD.6. Normalisasi posisi

Agar lebih mudah, kita definisikan:

$$Y_\tau = X_{t+\tau} - X_E.$$

Maka:

$$Y_0 = 0.$$

Batasnya:

$$-a < Y_t < b.$$

Dengan:

$$-a = \ln \frac{P_{SL}}{P_E}$$

dan:

$$b = \ln \frac{P_{TP}}{P_E}.$$

AD.7. Trade sekarang menjadi masalah first-passage

Kita definisikan waktu mencapai TP:

$$\tau_{TP} = \inf \{ \tau > 0 : Y_\tau = b \}.$$

Dan waktu mencapai SL:

$$\tau_{SL} = \inf \{ \tau > 0 : Y_\tau = -a \}.$$

Kemudian:

$$\tau_{exit} = \min(\tau_{TP}, \tau_{SL}).$$

AD.8. Ini adalah inti Tahap AD

Kita tidak lagi bertanya:

$$P(P_T > P_E)?$$

Tetapi:

$$P(\tau_{TP} < \tau_{SL}).$$

Artinya:

berapa probabilitas TP tercapai sebelum SL?

Ini jauh lebih relevan terhadap trading.

AD.9. Menurunkan probabilitas TP sebelum SL

Misalkan:

$$Y_t = \mu t + \sigma W_t.$$

Definisikan:

$$p(y) = P_y(\tau_b < \tau_{-a}).$$

Dengan boundary condition:

$$p(-a) = 0$$

dan:

$$p(b) = 1.$$

Untuk diffusion process, fungsi $p(y)$ memenuhi backward Kolmogorov equation:

$$\mu p'(y) + \frac{1}{2} \sigma^2 p''(y) = 0.$$

AD.10. Menyelesaikan persamaan

Kita punya:

$$\frac{1}{2} \sigma^2 p'' + \mu p' = 0.$$

Misalkan:

$$q = p'.$$

Maka:

$$\frac{1}{2}\sigma^2 q' + \mu q = 0.$$

Sehingga:

$$q' = -\frac{2\mu}{\sigma^2}q.$$

Solusinya:

$$q(y) = C e^{-2\mu y/\sigma^2}.$$

Integrasi:

$$p(y) = A + B e^{-2\mu y/\sigma^2}.$$

Dengan boundary condition kita memperoleh:

$$p(y) = \frac{1 - e^{-2\mu(y+a)/\sigma^2}}{1 - e^{-2\mu(a+b)/\sigma^2}}.$$

Karena entry berada pada:

$$y = 0,$$

maka:

$$p_{TP} = \frac{1 - e^{-2\mu a/\sigma^2}}{1 - e^{-2\mu(a+b)/\sigma^2}}.$$

Ini adalah probabilitas TP tercapai **sebelum** SL.

AD.11. Kasus tanpa drift

Jika:

$$\mu \rightarrow 0,$$

kita tidak boleh langsung memasukkan $\mu = 0$ ke bentuk tersebut karena menghasilkan $0/0$.

Gunakan limit:

$$e^{-x} \approx 1 - x.$$

Maka:

$$1 - e^{-2\mu a/\sigma^2} \approx \frac{2\mu a}{\sigma^2}.$$

Dan:

$$1 - e^{-2\mu(a+b)/\sigma^2} \approx \frac{2\mu(a+b)}{\sigma^2}.$$

Sehingga:

$$p_{TP} = \frac{a}{a+b}$$

untuk:

$$\mu = 0.$$

Menariknya, tanpa drift, probabilitas ditentukan oleh posisi relatif terhadap kedua boundary.

AD.12. Interpretasi finansial

Misalkan:

$$a = 1R$$

dan:

$$b = 2R.$$

Jika:

$$\mu = 0,$$



maka:

$$p_{TP} = \frac{1}{3}.$$

Sedangkan:

$$p_{SL} = \frac{2}{3}.$$

Jadi trade dengan reward:risk:

$$2 : 1$$

tidak otomatis memiliki probabilitas profit besar.

AD.13. Expected value trade

Sekarang kita dapat membangun expected value.

Misalkan:

$$R_{TP} = b$$

dan:

$$R_{SL} = a.$$

Untuk long:

$$EV = p_{TP}R_{TP} - (1 - p_{TP})R_{SL} - C.$$

dengan:

$$C = C_{entry} + C_{exit} + C_{slippage} + C_{funding}.$$

AD.14. Breakeven probability

Abaikan biaya terlebih dahulu.

Trade profitable jika:

$$pR_{TP} - (1 - p)R_{SL} > 0.$$

Maka:

$$pR_{TP} > R_{SL} - pR_{SL}.$$

Sehingga:

$$p(R_{TP} + R_{SL}) > R_{SL}.$$

Jadi:

$$p_{BE} = \frac{R_{SL}}{R_{TP} + R_{SL}}.$$

AD.15. Contoh

Jika:

$$R_{SL} = 1$$

dan:

$$R_{TP} = 2,$$

maka:

$$p_{BE} = \frac{1}{3} \approx 33.33\%.$$

Jadi kita tidak membutuhkan win rate 50%.

Tetapi setelah biaya:

$$p_{BE} > 33.33\%.$$

AD.16. Dengan biaya

Misalkan biaya dalam unit risk:

c .

Maka:

$$pR_{TP} - (1-p)R_{SL} - c > 0.$$

Sehingga:

$$p(R_{TP} + R_{SL}) > R_{SL} + c.$$

Maka:

$$p_{BE} = \frac{R_{SL} + c}{R_{TP} + R_{SL}}.$$

AD.17. Ini menghasilkan trade filter

Sistem kita dapat mengatakan:

$$p_{TP} > p_{BE}$$

sebagai salah satu syarat minimum.

Tetapi saya menyarankan jangan berhenti di sana.

Kita juga perlu:

$$EV > 0$$

dan:

$$\textit{Risk constraints}$$

serta:

$$\textit{Liquidity constraints.}$$

AD.18. Sekarang kita dapat mengoptimalkan TP

Misalkan entry dan SL sudah ditentukan.

Kita punya kandidat:

$$TP_1, TP_2, \dots, TP_K.$$

Untuk masing-masing:

$$b_k = \ln \frac{TP_k}{P_E}.$$

Hitung:

$$p_k = P(\tau_{TP_k} < \tau_{SL}).$$

Kemudian:

$$EV_k = p_k R_k - (1 - p_k) R_{SL} - C_k.$$

Maka:

$$TP^* = \arg \max_{TP_k} EV_k.$$

AD.19. Jadi TP bukan fixed 2%

Ini perubahan besar.

Sistem tidak berkata:

TP = 2%.

Melainkan:

$$TP^* = f(\mu_t, \sigma_t, regime_t, liquidity_t, microstructure_t, SL).$$

AD.20. Optimal SL juga

Sekarang misalkan TP tetap.

Kita punya kandidat:

$$SL_1, SL_2, \dots, SL_J.$$

Untuk setiap:

$$SL_j,$$

kita hitung:

$$a_j = \ln \frac{P_E}{SL_j}.$$

Kemudian:

$$p_j = P(\tau_{TP} < \tau_{SL_j}).$$

Dan:

$$EV_j = p_j R_{TP} - (1 - p_j) R_{SL_j} - C_j.$$

Maka:

$$SL^* = \arg \max_{SL_j} EV_j.$$

AD.21. Tetapi ada constraint risk

Misalkan modal risk maksimum:

$$R_{\max} = \$1.$$

Maka tidak semua SL boleh dipilih.

Position size:

$$Q_j = \frac{R_{\max}}{D_j + C_j^{\text{risk}}},$$



dengan:

$$D_j = |P_E - P_{SL_j}|.$$

Lebih realistis:

$$Q_j = \frac{R_{\max}}{D_j + E[Slippage_{SL_j}] + Fees}.$$

AD.22. Ini sangat penting

Kita tidak memilih:

$$SL$$

terlebih dahulu kemudian memaksakan:

$$Q.$$

Kita melakukan:

$$(SL, TP, Q)$$

sebagai satu paket keputusan.

AD.23. Trade candidate menjadi vector

Kita definisikan:

$$\theta = (P_E, P_{SL}, P_{TP}, Q, \tau_{\max}).$$

Kemudian objective:

$$J(\theta) = EV(\theta) - \lambda_R Risk(\theta) - \lambda_T TimeCost(\theta).$$

AD.24. Time cost

Trade yang membutuhkan 3 hari berbeda dari trade yang membutuhkan 30 menit.

Misalkan:

$$E[\tau_{exit}]$$

adalah expected holding time.

Kita dapat memberikan penalty:

$$C_{time} = \kappa E[\tau_{exit}].$$

Sehingga:

$$J = EV - \kappa E[\tau_{exit}].$$

Ini penting untuk strategi short-term.

AD.25. Mengapa holding time penting?

Misalkan dua trade:

Trade A

$$EV = \$0.10$$

dengan:

$$E[T] = 10 \text{ menit.}$$

Trade B

$$EV = \$0.20$$

dengan:

$$E[T] = 24 \text{ jam.}$$

Trade B memiliki EV absolut lebih besar.

Tetapi capital efficiency bisa jauh lebih buruk.

AD.26. Opportunity cost modal

Kita dapat mendefinisikan:

$$EV_{rate} = \frac{EV}{E[T]}.$$

atau secara annualized:

$$EV_{annualized} \approx \frac{EV}{E[T]} \times T_{year}.$$

Namun hati-hati: annualization agresif dapat menyesatkan jika trade tidak benar-benar dapat direplikasi.

AD.27. Expected holding time

Untuk proses diffusion dengan dua absorbing boundaries, expected exit time:

$$m(y) = E_y[\tau_{exit}]$$

memenuhi:

$$\mu m'(y) + \frac{1}{2} \sigma^2 m''(y) = -1.$$

Boundary:

$$m(-a) = m(b) = 0.$$

Pada:

$$y = 0$$

kita mendapatkan expected holding time.

Untuk:

$$\mu = 0,$$

hasil sederhananya:

$$E[\tau_{exit}] = \frac{ab}{\sigma^2}.$$

Ini sangat menarik.

AD.28. Interpretasi

Jika:

$$a$$

dan:

$$b$$

semakin jauh, holding time meningkat.

Jika:

$$\sigma$$

semakin tinggi:

$$E[T] \downarrow .$$

Artinya volatility tinggi membuat boundary lebih cepat tersentuh.

Tetapi ini **tidak berarti volatility tinggi selalu bagus**.

Karena:

$$SL$$

juga lebih mudah terkena.

AD.29. Maximum holding time

Di dunia nyata kita tidak ingin trade menggantung selamanya.

Definisikan:

$$T_{max}.$$

Jika:

$$\tau_{exit} > T_{max},$$

maka:

$$\boxed{Forced\ Exit.}$$

Ini menghasilkan boundary ketiga:

$$\boxed{TP, SL, TimeStop.}$$

AD.30. Maka trade menjadi three-barrier problem

Kita memiliki:

$$P_{TP},$$

$$P_{SL},$$

dan:

$$T_{max}.$$

Trade berakhir ketika salah satu tercapai.

Secara matematis:

$$\boxed{\tau = \min(\tau_{TP}, \tau_{SL}, T_{max}).}$$

AD.31. Probability vector

Kita sekarang ingin:

$$\mathbf{p} = \begin{bmatrix} P(TP) \\ P(SL) \\ P(TimeStop) \end{bmatrix}.$$

Dengan:

$$P(TP) + P(SL) + P(TimeStop) = 1.$$

AD.32. Mengapa ini lebih baik daripada win rate?

Karena kita tidak lagi hanya menyimpan:

win/loss.

Kita menyimpan:

$P(TP), \quad P(SL), \quad P(TimeStop).$

Dan juga:

$E[T].$

Sehingga setiap trade memiliki probabilistic profile.

AD.33. Trade quality score

Kita dapat mendefinisikan:

$$Q_t = EV_t - \lambda_1 Risk_t - \lambda_2 Time_t.$$

Tetapi saya lebih menyukai mempertahankan komponen-komponen ini terpisah.

Jangan terlalu cepat membuat satu "magic score".

AD.34. Entry optimization

Sekarang kita kembali ke entry.

Misalkan current mid-price:

$$M_t.$$

Kita memiliki candidate entry:

$$P_{E,1}, P_{E,2}, \dots, P_{E,K}.$$

Untuk setiap entry:

$$P_{E,k},$$

kita harus menghitung ulang:

$$p_{fill,k},$$

$$p_{TP,k},$$

$$EV_k.$$

Maka:

$$P_E^* = \arg \max_{P_{E,k}} EV_k.$$

AD.35. Ini menggabungkan AC + AD

Pada AC:

$$P_{E,k} \rightarrow p_{fill,k}.$$

Pada AD:

$$P_{E,k} \rightarrow p_{TP,k}, EV_k.$$

Sehingga:

$$EV_k = p_{fill,k} [p_{TP|fill,k} R_{TP} - (1 - p_{TP|fill,k}) R_{SL}] - C_k.$$

Ini jauh lebih realistis.

AD.36. Conditional probability chain

Kita dapat menulis:

$$P(TP) = P(Fill)P(TP | Fill).$$

Jadi:

$$P(TP) = p_{fill} \cdot p_{TP|fill}.$$

Demikian pula:

$$P(SL) = p_{fill} \cdot p_{SL|fill}.$$

AD.37. Tetapi ada satu kasus lagi

Jika order **tidak filled**, tidak ada trade.

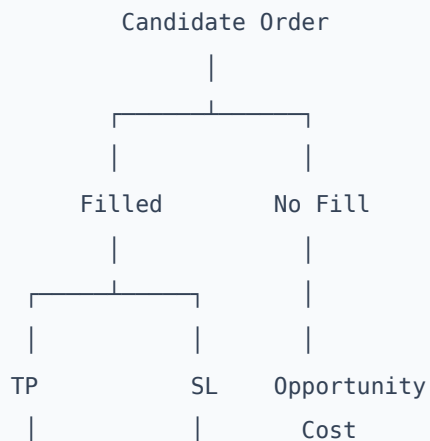
Maka:

$$P(no\ trade) = 1 - p_{fill}.$$

Jadi expected value keseluruhan:

$$EV = p_{fill}EV_{trade} - C_{opportunity}.$$

AD.38. Ini menghasilkan decision tree



Ini secara konseptual sangat penting untuk backtest kita.

AD.39. Market order

Untuk market order:

$$p_{fill} \approx 1$$

tetapi:

$$C_{execution}$$

lebih besar.

Untuk limit order:

$$p_{fill} < 1$$

tetapi:

$$C_{execution}$$

lebih kecil.

Jadi sistem dapat membandingkan:

$$EV_{market} \text{ vs } EV_{limit}$$

AD.40. Dynamic entry

Misalkan sekarang:

$$EV_{limit} = 0.08R$$

sedangkan:

$$EV_{market} = 0.12R.$$

Maka:

market

lebih baik.

Tetapi 20 menit kemudian:

$$EV_{limit} = 0.15R$$

dan:

$$EV_{market} = 0.10R.$$

Sistem seharusnya beralih:

market → *limit*.

Ini adalah **execution adaptation**.

AD.41. Sekarang kita masuk ke posisi yang sudah terbuka

Ini langsung menjawab ide awalmu tentang:

"jika floating profit tinggi, apakah TP dinaikkan?"

Misalkan entry:

$$P_E.$$

Harga sekarang:

$$P_t > P_E.$$

Kita tidak boleh mengatakan:

"Sudah profit 2%, naikkan TP."

Sebaliknya kita menghitung ulang:

$$EV(TP_k, SL_j | F_t).$$

AD.42. Re-optimization

Pada setiap evaluation interval:

$$t = t_0, t_1, t_2, \dots$$

kita hitung:

$$\theta_t^* = \arg \max_{\theta} J(\theta | F_t).$$

Dengan:

$$\theta = (SL, TP, PositionSize, ExitTime).$$

AD.43. Dynamic stop loss

Misalkan:

$$SL_{old} = P_E.$$

Kita menemukan:

$$SL^* > P_E.$$

Maka kita dapat melakukan:

$$SL_{new} = SL^*.$$

Ini berarti kita mengunci profit.

Tetapi:

$$SL^*$$

tidak boleh hanya berdasarkan floating profit.

Harus memenuhi:

$$EV(SL^*, TP) > EV(SL_{old}, TP).$$

AD.44. Dynamic TP

Jika:

$$TP^* > TP_{old},$$

kita naikkan TP hanya jika:

$$EV(TP^*) > EV(TP_{old}).$$

Jadi trailing TP menjadi hasil optimasi.

AD.45. Tetapi ada masalah penting: sunk cost

Setelah posisi terbuka, kita tidak boleh lagi menggunakan:

$$P_E$$

sebagai satu-satunya dasar keputusan.

Keputusan optimal harus berdasarkan:

$$\text{future conditional value.}$$

Artinya:

"Jika saya **sudah memiliki posisi ini sekarang**, apa tindakan terbaik dari titik ini?"

Ini adalah masalah **stochastic control**.

AD.46. State posisi

Kita dapat mendefinisikan state:

$$S_t = (P_t, P_E, SL_t, TP_t, Q_t, \mu_t, \sigma_t, Regime_t, M_t).$$

Kemudian action:

$$a_t = (Hold, Reduce, Close, MoveSL, MoveTP).$$

AD.47. Objective continuation value

Untuk action a :

$$V(S_t, a) = E[Future PnL - Costs \mid S_t, a].$$

Kemudian:

$$a_t^* = \arg \max_a V(S_t, a).$$

Ini adalah awal dari **dynamic programming**.

AD.48. Bellman equation

Jika kita ingin benar-benar membangun model matematis lanjut:

$$V(S_t) = \max_{a_t} \{r(S_t, a_t) + E[V(S_{t+1}) \mid S_t, a_t]\}.$$

Dengan discount factor γ :

$$V(S_t) = \max_a [r_t + \gamma E[V(S_{t+1}) \mid S_t, a]].$$



Ini nantinya dapat menjadi jembatan ke:

- stochastic control,
- dynamic programming,
- reinforcement learning.

Tetapi **belum kita gunakan sebagai core engine**.

AD.49. Mengapa saya tidak menyarankan RL sekarang?

Karena kita sudah memiliki struktur matematis yang lebih interpretable:

$$\mu, \sigma, P(TP), P(SL), EV, Risk, Liquidity.$$

Kalau langsung menggunakan RL, kita berisiko mendapatkan:

agent menghasilkan profit.

tetapi tidak mengetahui:

mengapa agent melakukan tindakan tersebut.

Untuk penelitian yang ingin memiliki fondasi matematis kuat, lebih baik:

$$\boxed{\text{Analytical model} \rightarrow \text{Optimization} \rightarrow \text{RL optional}.}$$

AD.50. Market regime masuk ke AD

Misalkan regime:

$$z_t \in \{Bull, Bear, Sideways, HighVol, LowVol\}.$$

Maka:

$$\mu = \mu(z_t),$$

$$\sigma = \sigma(z_t).$$

Akibatnya:

$$p_{TP} = p_{TP}(a, b, \mu(z_t), \sigma(z_t)).$$

Jadi TP/SL optimal berbeda untuk setiap regime.

AD.51. Contoh konseptual

Regime A

$$\mu > 0, \quad \sigma \text{ rendah.}$$

Model mungkin menghasilkan:

$$TP^* = 3R.$$

Regime B

$$\mu > 0, \quad \sigma \text{ sangat tinggi.}$$

Mungkin:

$$TP^* = 1.5R.$$

Karena walaupun upside besar, probabilitas SL juga meningkat.

AD.52. Market factor juga masuk

Kita sebelumnya ingin sistem tidak hanya crypto.

Untuk saham internasional, state kita dapat memasukkan:

$$F_t = \begin{bmatrix} R_{SPX} \\ R_{NDX} \\ R_{DXY} \\ R_{Gold} \\ R_{UST} \\ \dots \end{bmatrix}.$$

Kemudian:

$$\mu_{i,t} = \alpha_i + \beta_i^\top F_t.$$

Sehingga:

$$p_{TP,i} = f(\mu_{i,t}, \sigma_{i,t}, F_t, \text{Microstructure}_t).$$

AD.53. Contoh BTC

Misalnya kita ingin trading altcoin A.

State:

$$S_t = [r_A, r_{BTC}, r_{ETH}, r_{DXY}, r_{Gold}, OBI_A, OFI_A, \sigma_A, \text{Regime}].$$

Maka:

$$\mu_A = \alpha_A + \beta_{BTC} r_{BTC} + \beta_{ETH} r_{ETH} + \beta_{DXY} r_{DXY} + \beta_{Gold} r_{Gold} + \theta^\top \text{Microstructure}.$$

AD.54. Untuk saham internasional

Misalkan kita ingin trading:

$$AAPL.$$

State dapat berupa:

$$S_t = [r_{AAPL}, r_{SPX}, r_{NDX}, r_{DXY}, r_{Gold}, r_{UST}, OBI, OFI, Volatility].$$

Model:

$$\mu_{AAPL} = \alpha + \beta_{SPX} r_{SPX} + \beta_{NDX} r_{NDX} + \beta_{DXY} r_{DXY} + \beta_{Gold} r_{Gold} + \beta_{UST} r_{UST} + \dots.$$

Jadi **engine AD** tidak perlu tahu apakah underlying-nya crypto atau saham.

AD.55. Asset-agnostic architecture

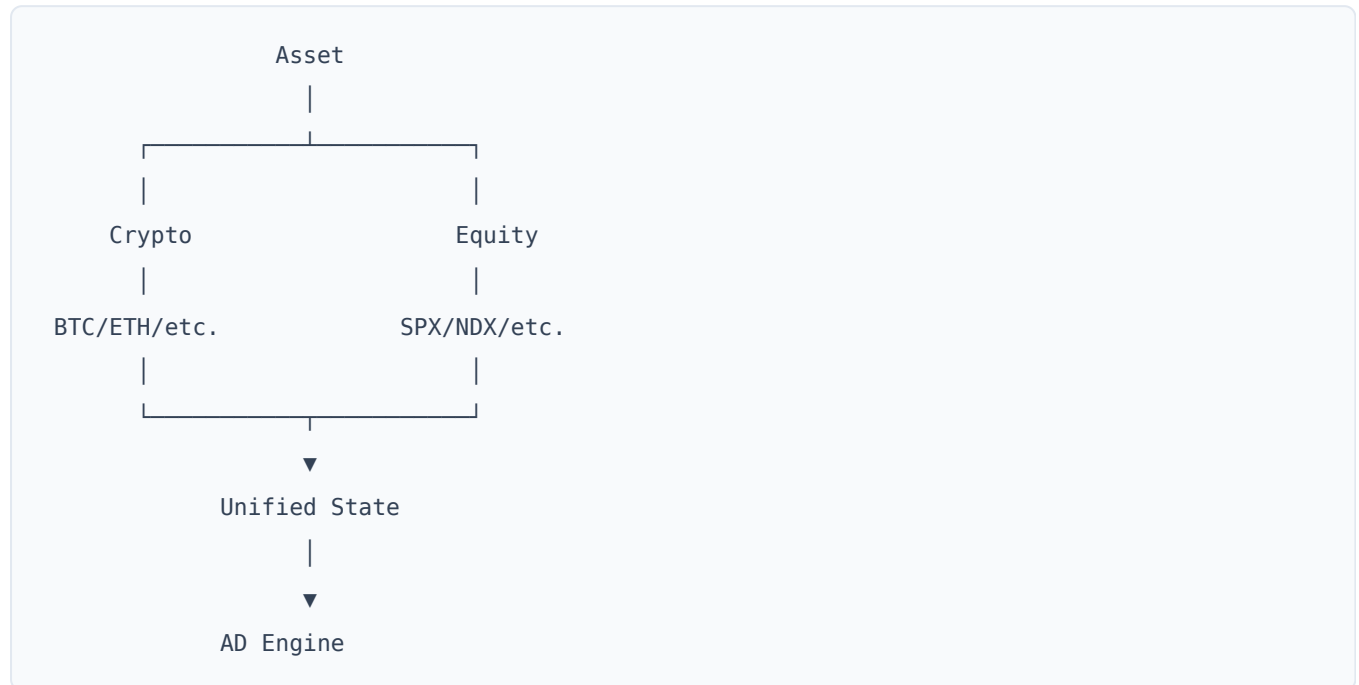
Kita dapat mendefinisikan:

$$\text{AssetState}_t = [\text{Return}, \text{FactorExposure}, \text{Volatility}, \text{Liquidity}, \text{Microstructure}, \text{Regime}].$$

Kemudian seluruh engine menggunakan:

$$AssetState_t.$$

Dengan demikian:



AD.56. Sekarang kita dapat mendefinisikan Trade Opportunity

Sebuah trade candidate:

$$T = (asset, side, entry, SL, TP, size, TTL).$$

Trade candidate diterima hanya jika:

$$EV(T) > 0.$$

Dengan constraint:

$$Risk(T) \leq R_{\max},$$

$$Liquidity(T) \geq L_{\min},$$

$$p_{fill} \geq p_{\min}.$$

AD.57. Opportunity set

Misalkan kita memiliki N aset dan K konfigurasi trade per aset.

Maka:

$$T_t = \{T_{1,1}, T_{1,2}, \dots, T_{N,K}\}.$$

Kita kemudian menghitung:

$$EV(T_{i,k}).$$

AD.58. Ini mengarah kembali ke optimasi kombinatorial

Sekarang kita bisa memiliki:

$$x_{i,k} \in \{0, 1\}$$

dengan:

$$x_{i,k} = 1$$

jika trade candidate k pada asset i dipilih.

Constraint:

$$\sum_{i,k} x_{i,k} \leq N_{max}.$$

Objective:

$$\max_x \sum_{i,k} EV_{i,k} x_{i,k} - \lambda Risk(x).$$

Ini sudah menjadi kandidat kuat untuk:

$$QUBO.$$

AD.59. Jadi sekarang QUBO kita semakin menarik

Sebelumnya:

x_i = asset selected.

Sekarang:

$x_{i,k}$ = asset + strategy configuration selected.

Misalnya:

BTC

- |— Long, TP=1R, SL=1R
- |— Long, TP=2R, SL=1R
- |— Long, TP=3R, SL=1R
- └— Short, TP=2R, SL=1R

ETH

- |— Long, TP=1R, SL=1R
- |— Long, TP=2R, SL=1R
- ...

Quantum optimizer memilih kombinasi terbaik.

AD.60. Ini merupakan perkembangan besar

Awalnya kita memiliki:

Portfolio Optimization

Sekarang menjadi:

Trade Opportunity Optimization.

Decision variable bukan lagi hanya:

weight.

Tetapi:

$(asset, side, entry, SL, TP, size, execution)$.

AD.61. Namun jangan langsung mengoptimalkan semuanya

Ini penting secara engineering.

Kita sebaiknya memiliki hierarchy:

Level 1

Forecast

Level 2

Opportunity

Level 3

Portfolio

Level 4

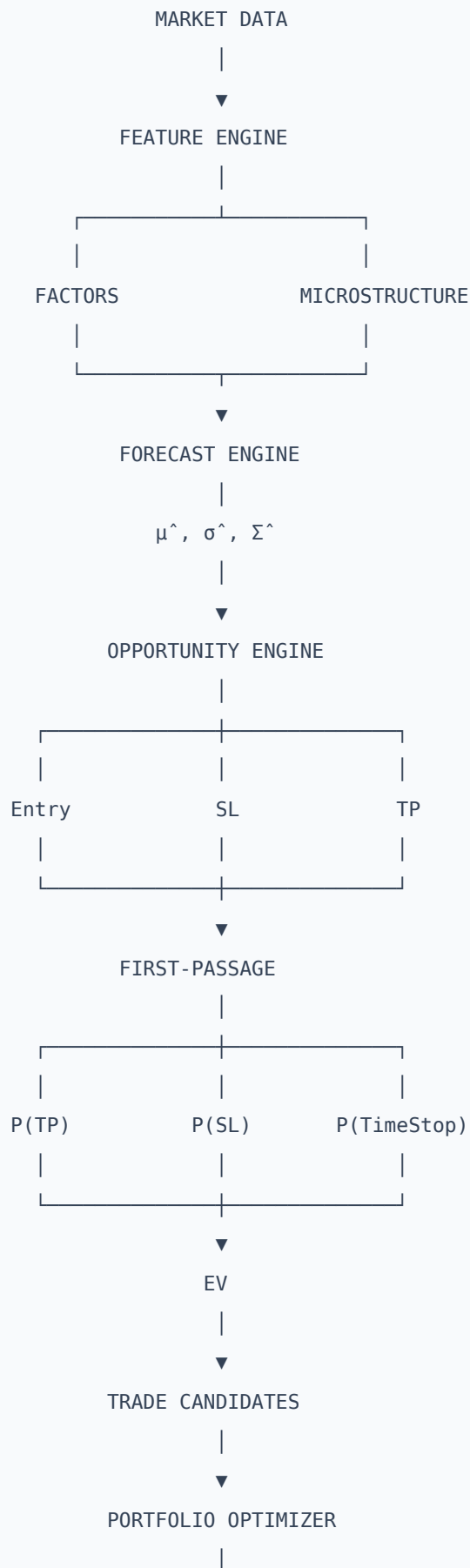
Execution

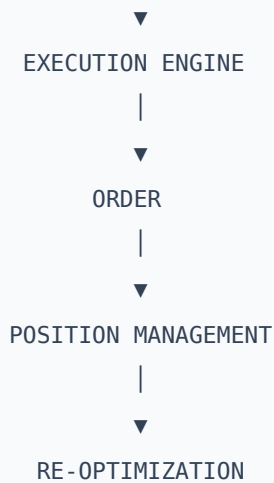
Level 5

Position Management.

Karena jika semua variabel dilempar ke satu optimizer, dimensionality akan meledak.

AD.62. Pipeline final setelah AD





AD.63. Formula inti Tahap AD

Kalau harus diringkas menjadi satu rangkaian matematis:

1. Conditional process

$$dX_t = \mu_t dt + \sigma_t dW_t$$

2. Barrier

$$-a < X_t - X_E < b$$

3. First-passage probability

$$p_{TP} = \frac{1 - e^{-2\mu a/\sigma^2}}{1 - e^{-2\mu(a+b)/\sigma^2}}$$

4. Expected value

$$EV = p_{TP} R_{TP} - (1 - p_{TP}) R_{SL} - C$$

5. Optimal configuration

$$\theta^* = \arg \max_{\theta} EV(\theta)$$

dengan:

$$\theta = (Entry, SL, TP, Q, TTL).$$

AD.64. Dan inilah koneksi langsung dengan target \$1 risk

Kita ingin:

$$R_{\max} = \$1.$$

Untuk setiap kandidat:

$$\theta_k,$$

hitung:

$$Q_k = \frac{\$1}{D_{SL,k} + E[Slippage_{SL,k}] + Fees_k}.$$

Kemudian:

$$R_{TP,k} = Q_k D_{TP,k}.$$

Dan:

$$EV_k = p_{TP,k} R_{TP,k} - (1 - p_{TP,k}) \$1 - Costs_k.$$

Maka sistem memilih:

$$\theta^* = \arg \max_k EV_k.$$

Dengan kata lain:

\$1 bukan target profit. \$1 adalah budget risiko maksimum per trade.

Ini perbedaan yang sangat penting.

AD.65. Bahkan kita dapat mendefinisikan "quality of opportunity"

Misalkan:

$$EV^* > 0.$$

Tetapi kita dapat membandingkan terhadap risk:

$$\text{Opportunity Ratio} = \frac{EV^*}{R_{\max}}.$$

Misalnya:

$$EV^* = 0.15R.$$

Berarti:

$$OR = 0.15.$$

Semakin besar semakin menarik, dengan catatan estimasi probabilitasnya reliable.

AD.66. Tetapi ada satu hal yang harus kita cegah

Jangan sampai sistem memilih:

$$TP = 100R$$

hanya karena model memberikan sedikit probabilitas untuk mencapainya.

Misalnya:

$$p = 0.01, \quad TP = 100R.$$

Secara naïf:

$$EV = 0.01(100) - 0.99(1) = 0.01R.$$

Terlihat positif.

Tetapi estimasi tail probability seperti itu sangat rentan estimation error.

Maka kita membutuhkan:

$P(\text{Probability calibration})$

dan:

tail constraints.

Ini akan menjadi bagian penting tahap berikutnya.

AD.67. AD menghasilkan fondasi untuk sesuatu yang jauh lebih besar

Sekarang kita telah memiliki tiga lapisan probabilistik:

$P(Fill)$

$P(TP | Fill)$

$P(SL | Fill)$

sehingga:

$P(TP) = P(Fill)P(TP | Fill).$

Dan akhirnya:

$EV = P(TP)G - P(SL)L - C.$

Ini adalah **trade-level expected value engine**.

AD.68. Hubungan AD dengan seluruh visi awalmu

Ide awalmu adalah:

"gunakan data historis, lihat aset terkait, pasang order book, tentukan TP/SL, lalu evaluasi posisi secara berkala."

Sekarang masing-masing sudah mempunyai fondasi matematis:

Ide awal	Engine matematis
Data historis	Forecast engine
BTC/ETH/Gold/DXY	Factor engine
Order book	Microstructure engine
Limit order	Fill probability
Harga entry	Entry optimization
SL	Barrier optimization
TP	Barrier optimization
\$1 risk	Position sizing
Floating profit	State update
Naikkan SL	Dynamic optimization
Naikkan TP	Dynamic optimization
Evaluasi berkala	Receding-horizon optimization
Banyak aset	Portfolio optimization
Kombinasi trade	QUBO
Short-term	First-passage/time horizon

Jadi kita sekarang **bukan sekadar membuat bot trading**.

Kita sedang membangun:

Probabilistic Multi-Asset Optimal Trading System

yang kemudian dapat mempunyai **quantum optimization layer** sebagai salah satu solver-nya.

AD.69. Langkah berikutnya

Setelah AD, ada satu masalah yang harus kita selesaikan sebelum sistem boleh dipercaya:

AD belum tahu apakah probabilitasnya benar.

Misalnya model mengatakan:

$$P(TP) = 0.72.$$

Apakah 72% benar-benar berarti sekitar 72 dari 100 trade serupa akan mencapai TP sebelum SL?

Kalau tidak, seluruh:

EV ,

TP^* ,

SL^* ,

bahkan QUBO kita bisa menjadi **garbage-in-garbage-out yang sangat matematis**.

Karena itu **Tahap AE** seharusnya kita dedikasikan untuk:

Probability Calibration, Statistical Validation & Backtest Integrity

yang mencakup **walk-forward validation, purged/embargoed time-series cross-validation, calibration curve, Brier score, log-loss, confidence interval, bootstrap, multiple-testing bias, data leakage, survivorship bias, look-ahead bias, dan Deflated/Probabilistic Sharpe Ratio**.

Ini menurut saya justru salah satu tahap paling krusial dalam keseluruhan proyek, karena setelah AE kita baru bisa mengatakan bahwa optimasi yang kita lakukan **memiliki bukti statistik**, bukan sekadar menghasilkan backtest yang terlihat bagus.